Gradient

What It Is, Why We Needed It, How We Reached It

1. Where the Problem Appears

By Lab 4, we already have:

- a model that produces an output
- a loss function that tells us how wrong the output is

But this still leaves a missing step:

> If the output is wrong, how exactly should the weights change?

Saying “reduce error” is not enough.

We need **direction** and **magnitude**.

This is where the gradient enters.

2. What the Gradient Is (Conceptual)

A **gradient** answers one precise question:

> “If I change this parameter slightly, how will the loss change?”

For a single parameter w:

$$\frac{\partial L}{\partial w}$$

This quantity tells us:

- sign → which direction to move
- magnitude → how strongly to move

Nothing more. Nothing less.

3. Why Subtraction Appears in Learning

If:

$$\frac{\partial L}{\partial w} > 0$$

Increasing w increases loss.

So we must **decrease** w.

If:

$$\frac{\partial L}{\partial w} < 0$$

Increasing w decreases loss.

So we must **increase** w.

This leads to the update rule:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

Where:

- η is the learning rate
- subtraction ensures loss decreases

This is not heuristic.

This is calculus.

4. Gradient in Linear Regression (MSE Case)

Model

$$\hat{y} = wx + b$$

Loss (MSE, single sample)

$$L = (\hat{y} - y)^2$$

Gradient with respect to weight

$$\frac{\partial L}{\partial w} = 2(\hat{y} - y)x$$

Key observation:

- gradient grows **linearly** with error
- large errors → large updates
- small errors → small updates

This behavior is stable and correct for regression.

5. Why Gradient Alone Is Not Enough

Now move to classification.

Output is not a number like salary.

Output is **confidence**.

Using MSE here causes a problem.

6. Gradient Behavior with MSE + Sigmoid

Sigmoid output

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss (incorrect choice)

$$L = (\hat{p} - y)^2$$

Gradient (simplified intuition)

$$\frac{\partial L}{\partial z} \propto (\hat{p} - y) \cdot \hat{p}(1 - \hat{p})$$

Critical term:

$$\hat{p}(1-\hat{p})$$

This term goes to **zero** when:

- p^≈0
- p^≈1

Meaning:

> When the model is very confident, the gradient dies.

Even if the prediction is **confidently wrong**.

This is catastrophic.

7. Why This Is Bad

Example:

True label:

$$y = 1$$

Prediction:

$$\hat{p} = 0.001$$

Model is very wrong and very confident.

But:

$$\hat{p}(1-\hat{p}) \approx 0$$

So gradient is almost zero.

Learning barely happens.

This means:

> The model does not strongly correct confident mistakes.

That violates our mistake cost logic.

8. Enter Cross-Entropy

Cross-entropy was introduced to fix **gradient behavior**, not just loss scale.

Binary cross-entropy

$$L = -\left[y \log(\hat{p}) + (1-y)\log(1-\hat{p})\right]$$

---

9. Gradient of Cross-Entropy + Sigmoid

When sigmoid and cross-entropy are combined:

$$\frac{\partial L}{\partial z} = \hat{p} - y$$

That's it.

No extra terms.

No vanishing factor.

10. Why This Is Perfect

Look at behavior:

- If prediction is correct and confident → small gradient
- If prediction is wrong and confident → **large gradient**
- If prediction is uncertain → moderate gradient

This exactly matches what we want learning to do.

11. Gradient Behavior Comparison

Summary Table

Aspect	MSE + Sigmoid	Cross-Entropy + Sigmoid
Gradient near 0 or 1	Vanishes	Strong
Penalizes confidence	No	Yes

Aspect	MSE + Sigmoid	Cross-Entropy + Sigmoid
Learning speed	Slow	Fast
Correct for classification	No	Yes
Used in practice	Rare	Standard

12. Final Mental Model

- Loss tells **how wrong**
- Gradient tells **how to fix**
- Activation shapes **output space**
- Loss + activation together control **learning dynamics**

This is why:

- Linear regression → MSE + linear output
- Logistic regression → Cross-entropy + sigmoid

Same gradient descent.

Different mathematics because the **question changed**.

> “Gradient is not a training trick.
>
> Gradient is the only mathematically valid way to reduce error.”